

Pekko Streams, Slick & Verteilte Systeme

Pekko Streams, Slick & Verteilte Systeme

Gliederung



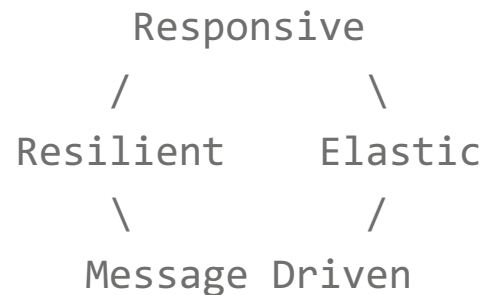
- 1. Pekko Streams: Grundlagen
- 2. Backpressure & Operatoren
- 3. Fehlerbehandlung & Graphs
- 4. Slick im Detail: Queries
- 5. Slick: DBIO, Transaktionen, Joins
- 6. Slick: Plain SQL & Streaming
- 7. Slick + Pekko Streams
- 8. Pekko Persistence & CQRS
- 9. Cluster & Sharding

Pekko Streams, Slick & Verteilte Systeme

Das Reactive Manifesto



- Leitbild für moderne, verteilte Systeme und die Grundlage von Pekko (2014)
- Vier Eigenschaften:
 - **Responsive:** reagiert zeitnah und vorhersehbar
 - **Resilient:** bleibt responsive auch bei Fehlern (Isolation, Supervision)
 - **Elastic:** skaliert mit der Last (hoch und runter)
 - **Message Driven:** asynchrone Nachrichten als Fundament (lose Kopplung)



Pekko Streams: Grundlagen

Pekko Streams: Grundlagen

Warum Streams?



- Probleme bei "alles auf einmal" laden/verarbeiten:
 - Datenmenge größer als der Speicher (Millionen DB-Zeilen, große Dateien, Events)
 - ein schneller Producer überrennt einen langsamen Consumer
- **Pekko Streams**: Daten fließen **elementweise** durch eine Pipeline
- Kernfeature: **Backpressure**, der Consumer bestimmt das Tempo
- Aufgebaut auf Aktoren (VL08), aber mit deklarativer, typsicherer API

Pekko Streams: Grundlagen

Source, Flow, Sink



- Drei Bausteine, wie ein Rohrsystem:
 - **Source[Out, Mat]**: produziert Elemente (Quelle)
 - **Flow[In, Out, Mat]**: transformiert Elemente
 - **Sink[In, Mat]**: konsumiert Elemente (Senke)
- Zusammengesteckt ergeben sie einen **RunnableGraph**
- `Mat` = der **materialisierte Wert** (Ergebnis beim Ausführen)

Source ~> Flow ~> Flow ~> Sink

Pekko Streams: Grundlagen

Ein einfacher Stream

- ``via(flow)`` hängt einen Verarbeitungsschritt an
- ``runWith(sink)`` **materialisiert** und startet den Stream
- Nichts läuft, bevor ``run*`` aufgerufen wird (wie bei Slick ``db.run``)

```
import org.apache.pekko.stream.scaladsl.{Source, Flow, Sink}

val source = Source(1 to 100)           // Source[Int, NotUsed]
val flow    = Flow[Int].map(_ * 2)      // Flow[Int, Int, NotUsed]
val sink    = Sink.foreach[Int](println) // Sink[Int, Future[Done]]

val done: Future[Done] = source.via(flow).runWith(sink)
```

Pekko Streams: Grundlagen

Materialization



- Ein Stream ist ein **Bauplan**, ``run*`` macht daraus eine laufende Pipeline
- Dabei entsteht der **materialisierte Wert**:
- ``Sink.fold`` materialisiert ``Future[Ergebnis]``, ``Sink.foreach`` ein ``Future[Done]``
- ``Keep.left/right/both`` steuert, welcher Mat-Wert durchgereicht wird

```
val sum: Future[Int] =  
    Source(1 to 100).runWith(Sink.fold(0)(_ + _))  
  
// Mat-Wert wählen mit Keep:  
val (control, done) =  
    source.viaMat(flow)(Keep.left).toMat(sink)(Keep.both).run()
```

Backpressure & Operatoren

Backpressure & Operatoren

Backpressure



- **Demand-getrieben:** der Consumer signalisiert, wie viele Elemente er will
- Der Producer sendet **nur so viel, wie nachgefragt wird**, kein Überlaufen
- Ist ein Schritt langsam, bremst sich die ganze Pipeline **automatisch** ab
- Implementiert nach dem **Reactive-Streams**-Standard (interoperabel)

Sink: "ich kann 4 verarbeiten" —demand—▶

Source: ◀— sendet höchstens 4 —

Backpressure & Operatoren

Puffer & OverflowStrategy



- Ein expliziter Puffer entkoppelt Producer und Consumer kurzfristig:
- **OverflowStrategy**, wenn der Puffer voll ist:
 - ``backpressure``: Producer ausbremsen (Default-Verhalten)
 - ``dropHead`` / ``dropTail``: ältestes / neuestes Element verwerfen
 - ``dropBuffer``: ganzen Puffer leeren
 - ``fail``: Stream mit Fehler beenden

```
import org.apache.pekko.stream.OverflowStrategy
```

```
source
```

```
  .buffer(size = 100, OverflowStrategy.backpressure) // Standard: bremst Producer  
  .map(slowProcessing)  
  .run()
```

- **conflate**: schneller Producer, langsamer Consumer, Elemente zusammenfassen statt bremsen
- **expand**: langsamer Producer, schneller Consumer, fehlende Werte auffüllen
- Beide verändern bewusst die Element-Rate, statt Backpressure durchzureichen

source

```
.conflate((acc, e) => acc + e) // fasst wartende Elemente zusammen  
.throttle(1, 1.second)  
.runForeach(println)
```

```
source.expand(last => Iterator.continually(last)) // wiederholt das letzte Element
```

Backpressure & Operatoren

Stages entkoppeln: **async**

- Standardmäßig werden Operatoren **fusioniert** (ein Akteur, ein Thread)
- ``.async`` setzt eine **asynchrone Grenze**, Stages laufen dann **parallel** (Pipelining):
- Die Grenze ist ein **bounded buffer** mit Backpressure zwischen den Stages
- Sinnvoll, wenn einzelne Stages teuer sind und sich parallelisieren lassen

source

```
.map(stage1).async    // eigener Akteur/Thread  
.map(stage2).async    // läuft nebenläufig zu stage1  
.runWith(sink)
```

Backpressure & Operatoren

Wichtige Operatoren



- **map / filter / collect:** transformieren, filtern (wie auf Collections)
- **grouped(n) / sliding(n):** Elemente bündeln (Batching)
- **scan / fold:** Zustand über den Stream aufakkumulieren
- **throttle(n, per):** Rate begrenzen (max. n Elemente pro Zeiteinheit)
- **take / drop / takeWhile:** begrenzen
- **mapConcat:** ein Element zu mehreren (flatMap auf Collection)

```
Source(1 to 1000)
  .filter(_ % 2 == 0)
  .grouped(50)           // Seq[Int] in Batches
  .throttle(1, 1.second) // ein Batch pro Sekunde
  .runForeach(batch => println(batch.sum))
```

Backpressure & Operatoren

mapAsync: I/O im Stream



- Für asynchrone Arbeit pro Element (DB, HTTP) → `mapAsync`:
- **mapAsync(n)**: bis zu `n` Futures parallel, **Reihenfolge bleibt** erhalten
- **mapAsyncUnordered(n)**: emittiert, sobald ein Future fertig ist (schneller, ungeordnet)
- Backpressure begrenzt automatisch die Anzahl offener Futures

```
Source(userIds)  
  .mapAsync(parallelism = 4)(id => httpClient.fetchUser(id)) // Future pro Element  
  .runWith(Sink.seq)
```

Fehlerbehandlung & Graphs

Fehlerbehandlung & Graphs

Fehler im Stream behandeln

- Standard: eine Exception **beendet** den Stream (der Mat-Future schlägt fehl)
- **recover** / **recoverWithRetries**: Fehler in ein letztes Element oder eine Ersatz-Source wandeln
- **Supervision-Strategie** pro Stream (Resume / Restart / Stop):

```
source
  .map(riskyParse)
  .recover { case _: NumberFormatException => -1 }    // letztes Element

val decider: Supervision.Decider = {
  case _: ArithmeticException => Supervision.Resume  // Element überspringen
  case _                      => Supervision.Stop
}
source.via(flow).withAttributes(
  ActorAttributes.supervisionStrategy(decider)
)
```

Fehlerbehandlung & Graphs

Quellen neu starten: RestartSource

- Bricht eine Quelle ab (z.B. Verbindung weg), startet `RestartSource` sie mit Backoff neu:

```
import org.apache.pekko.stream.scaladsl.RestartSource
import org.apache.pekko.stream.RestartSettings
import scala.concurrent.duration._

val settings = RestartSettings(
  minBackoff    = 1.second,
  maxBackoff    = 30.seconds,
  randomFactor  = 0.2           // streut die Wartezeit (gegen Thundering Herd)
)

val resilient: Source[Event, NotUsed] =
  RestartSource.withBackoff(settings) { () =>
    connectToEventStream()      // wird bei jedem Abbruch erneut aufgerufen
  }
```

Fehlerbehandlung & Graphs

Graphs: Fan-out & Fan-in

- Lineare Pipelines reichen oft nicht, **Verzweigungen** über die GraphDSL:
 - **Broadcast:** ein Eingang, mehrere Ausgänge (fan-out)
 - **Merge:** mehrere Eingänge, ein Ausgang (fan-in)
 - **Balance / Zip / Concat:** verteilen, paaren, aneinanderhängen
- Einfache Fälle ohne GraphDSL:

```
val merged = sourceA.merge(sourceB)           // zwei Quellen zusammenführen
val tee     = source.alsoTo(auditSink)         // Nebenabzweig (z.B. Logging)
```

Fehlerbehandlung & Graphs

GraphDSL: Broadcast & Merge

```
val graph = RunnableGraph.fromGraph(GraphDSL.create() { implicit b =>
  val bcast = b.add(Broadcast[Int])(2)    // 1 Eingang -> 2 Ausgänge
  val merge = b.add(Merge[Int])(2)        // 2 Eingänge -> 1 Ausgang
  val f1 = Flow[Int].map(_ + 10)
  val f2 = Flow[Int].map(_ * 100)

  Source(1 to 10) ~> bcast
                    bcast.out(0) ~> f1 ~> merge.in(0)
                    bcast.out(1) ~> f2 ~> merge.in(1)
                    merge ~> Sink.foreach(println)

  ClosedShape
})
graph.run()
```

Fehlerbehandlung & Graphs

Streaming-Response in Play



- Eine **Source** kann direkt als HTTP-Response gestreamt werden:
- `Ok.chunked(source)` sendet Elemente **nach und nach** an den Client
- Konstanter Speicher, auch bei riesigen Antworten
- Play stellt den nötigen **Materializer** bereit

```
def numbers = Action {  
    val source = Source(1 to 1000000).map(n => s"$n\n")  
    Ok.chunked(source)           // Transfer-Encoding: chunked  
}
```

Slick im Detail: Queries

Slick im Detail: Queries

Slick-Architektur



- **Profile** bindet Slick an eine konkrete DB (`H2Profile`, `PostgresProfile`)
- **TableQuery[T]** repräsentiert eine Tabelle als typsichere Collection
- **DBIOAction** beschreibt eine Operation (noch keine Ausführung)
- **Database + db.run(action)** führt aus → `Future[...]`

```
import slick.jdbc.H2Profile.api.*
```

```
val notes = TableQuery[NoteTable]  
val action = notes.filter(_.done === false).result // DBIOAction  
val result = db.run(action)                        // Future[Seq[Note]]
```

Slick im Detail: Queries

Tables & Column-Options

- **Column-Options:** `O.PrimaryKey`, `O.AutoInc`, `O.Length`, `O.Default`, `O.Unique`
- `\*` ist die **Default-Projektion** (Mapping Tabelle ↔ Case Class)
- `index(...)`, `foreignKey(...)` für Indizes und Beziehungen

```
class NoteTable(tag: Tag) extends Table[Note](tag, "notes") {  
  def id      = column[Long]("id", O.PrimaryKey, O.AutoInc)  
  def title   = column[String]("title", O.Length(255))  
  def done    = column[Boolean]("done", O.Default(false))  
  
  def * = (id, title, done).mapTo[Note]  
  def titleIdx = index("idx_note_title", title)  
}
```

Slick im Detail: Queries

Queries: filter, map, sortBy

- `===`, `!=`, `<`, `>`, `like`, `inSet` für Bedingungen
- `take`/`drop` = LIMIT/OFFSET, `sortBy` = ORDER BY

```
// WHERE done = false ORDER BY title LIMIT 10
```

```
notes.filter(_done === false)
      .sortBy(_.title.asc)
      .take(10)
      .result
```

```
// nur eine Spalte projizieren
```

```
notes.map(_.title).result           // Future[Seq[String]]
```

```
// einzelnes Ergebnis
```

```
notes.filter(_id === 1L).result.headOption // Future[Option[Note]]
```

Slick im Detail: Queries

Aggregationen & group by

- Aggregatfunktionen: `length`, `min`, `max`, `avg`, `sum`
- `groupBy` liefert pro Gruppe eine Sub-Query, die man weiter aggregiert

```
// COUNT / MAX / AVG
notes.length.result           // Future[Int]
notes.map(_.title).max.result  // Future[Option[String]]

// GROUP BY done, COUNT(*)
notes.groupBy(_.done)
  .map { case (done, group) => (done, group.length) }
  .result                     // Future[Seq[(Boolean, Int)]]
```

Slick im Detail: Queries

Joins



- **Applicative Join** (explizit, auch outer joins):
- **Monadic Join** (for-comprehension, nur inner):

```
val q = notes
  .join(categories).on(_.categoryId === _.id)    // inner join
  .map { case (n, c) => (n.title, c.name) }
```

```
notes.joinLeft(categories).on(_.categoryId === _.id) // Rep[Option[..]] rechts
```

```
val q = for {
  n <- notes
  c <- categories if n.categoryId === c.id
} yield (n.title, c.name)
```

Slick: DBIO, Transaktionen, Joins

Slick: DBIO, Transaktionen, Joins

Insert, Update, Delete

```
// INSERT mit generierter ID zurück
val withId = (notes returning notes.map(_.id)
              into ((n, id) => n.copy(id = id))) += note

// Batch-Insert
notes += Seq(note1, note2, note3)

// UPDATE (ganze Zeile oder einzelne Spalte)
notes.filter(_.id === 1L).update(updatedNote)
notes.filter(_.id === 1L).map(_.done).update(true)

// DELETE
notes.filter(_.id === 1L).delete
```

Slick: DBIO, Transaktionen, Joins

DBIO komponieren



- `DBIOAction` ist eine **Monade**, Aktionen lassen sich verketteten:
- `flatMap`/`map`/`for` für abhängige Schritte
- `DBIO.sequence(actions)` führt eine Liste von Aktionen aus

```
val combined: DBIO[Note] = for {  
  id    <- notes returning notes.map(_.id) += note  
  _     <- logTable += LogEntry(s"created $id")  
  saved <- notes.filter(_.id === id).result.head  
} yield saved
```

```
// ohne Abhängigkeit: DBIO.seq(a, b, c) / a andThen b
```

Slick: DBIO, Transaktionen, Joins

Transaktionen



- Mehrere Aktionen **atomar** ausführen mit ``.transactionally``:
- Schlägt ein Schritt fehl → **Rollback** der gesamten Aktion
- Funktioniert nur auf zusammengesetzten **DBIOActions**, nicht über mehrere ``db.run``

```
val transfer: DBIO[Unit] = (for {  
  _ <- accounts.filter(_.id === from).map(_.balance).update(newFrom)  
  _ <- accounts.filter(_.id === to).map(_.balance).update(newTo)  
} yield {}).transactionally
```

```
db.run(transfer)    // alles oder nichts
```

Slick: Plain SQL & Streaming

Slick: Plain SQL & Streaming

Plain SQL



- Wenn das DSL nicht reicht (DB-spezifisch, komplex): String-Interpolatoren:
- ``$minPrice`` wird als **gebundener Parameter** eingesetzt → kein SQL-Injection
- ``sql`".as[T]``, ``sqlu`"` (Update), ``tsql`"` (compile-time checked)

```
import slick.jdbc.H2Profile.api.*
```

```
val minPrice = 10.0
```

```
// Query mit typisiertem Ergebnis
```

```
val q = sql"select id, title from notes where id > $minPrice".as[(Long, String)]
```

```
// Update/DDL ohne Ergebnis
```

```
val u = sqlu"update notes set done = true where id = $id"
```

```
db.run(q)    // Future[Seq[(Long, String)]]
```

Slick: Plain SQL & Streaming

Streaming aus der DB



- ``db.run`` lädt **alles** in den Speicher, schlecht bei Millionen Zeilen
- ``db.stream`` liefert einen **Reactive-Streams-Publisher**, der Zeilen einzeln liefert:
- Konstanter Speicher: der Treiber holt die Zeilen häppchenweise (Cursor)
- Bei Postgres: ``.transactionally`` + ``.fetchSize`` nötig für echtes Server-Side-Streaming
- Genau die Brücke zu **Pekko Streams** (nächster Abschnitt)

```
val publisher: DatabasePublisher[Note] =  
    db.stream(notes.result)
```

Slick: Plain SQL & Streaming

Connection Pool & Konfiguration

- Slick nutzt **HikariCP** als Connection Pool (Produktivbetrieb):
- `numThreads` bestimmt die Parallelität der DB-Zugriffe
- In-Memory/Demo: `Database.forURL(...)` reicht (wie im Übungsprojekt)

```
mydb {  
  profile = "slick.jdbc.PostgresProfile$"  
  db {  
    url      = "jdbc:postgresql://localhost:5432/app"  
    user      = "app"  
    password  = "secret"  
    numThreads = 10      // Pool-Größe / AsyncExecutor  
    maxConnections = 10  
  }  
}
```

```
val db = DatabaseConfig.forConfig[JdbcProfile]("mydb").db
```

Slick + Pekko Streams

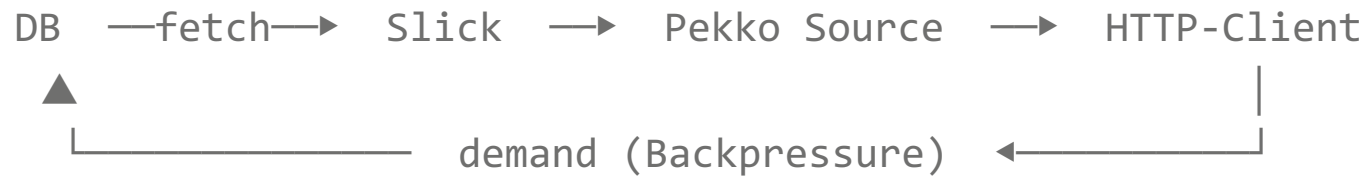
Slick + Pekko Streams

DB-Export als Stream

- ``db.stream`` → ``Source.fromPublisher`` → eine echte Pekko Source:
- DB-Zeilen fließen **einzel**n vom Treiber bis zum HTTP-Client
- Nie alles im Speicher, funktioniert auch bei Millionen Zeilen

```
def exportNotes = Action {  
  val publisher = db.stream(notes.result)           // DatabasePublisher[Note]  
  val source: Source[Note, NotUsed] =  
    Source.fromPublisher(publisher)  
  
  Ok.chunked(source.map(n => Json.toJson(n).toString + "\n"))  
}
```

- Der langsame Client bremst über Backpressure die **ganze Kette** ab:
- Liest der Client langsam, fragt Pekko weniger Zeilen nach
- Slick holt entsprechend weniger aus der DB (Cursor/fetchSize)
- Kombinierbar mit ``throttle``, ``grouped``, ``mapAsync`` für Transformation/Rate



Pekko Persistence & CQRS

Pekko Persistence & CQRS

Pekko Persistence: Event Sourcing



- Aktor-State ist flüchtig; Persistence speichert nicht den Zustand, sondern die **Events**
- `EventSourcedBehavior`: Command-Handler erzeugt Events, Event-Handler baut den State:
- Beim Start werden alle Events **replayed**

```
EventSourcedBehavior[Command, Event, State](  
  persistenceId = PersistenceId.ofUniqueId(id),  
  emptyState    = State(0),  
  commandHandler = (state, cmd) => cmd match {  
    case Deposit(a)          => Effect.persist(Deposited(a))  
    case GetBalance(replyTo) => Effect.none.thenReply(replyTo)(_.balance)  
  },  
  eventHandler = (state, evt) => evt match {  
    case Deposited(a) => state.copy(balance = state.balance + a)  
  }  
)
```

Pekko Persistence & CQRS

Persistence: Ablauf & Recovery



- **Schreiben:** Command → Event anhängen (append-only) → State aktualisieren
- **Recovery** (Start/Neustart): jüngsten Snapshot laden, dann nur neuere Events replaysen

Schreiben:

Command → commandHandler → persist(Event) → event_journal (append-only)
└─> eventHandler → neuer State

Recovery (Start):

snapshot (jüngster) → State
event_journal ab seqNr+1 → replay → aktueller State

Pekko Persistence & CQRS

Snapshots



- Problem: bei langen Historien wird das Replay **aller** Events langsam
- **Snapshot** = gespeicherter Zustand zu einer Sequenznummer
- Recovery lädt den jüngsten Snapshot und replayed nur die Events **danach**
- **Wichtig:** Snapshots sind nur eine **Optimierung**, die Events bleiben die Quelle der Wahrheit

```
EventSourcedBehavior[Command, Event, State](...)  
  .withRetention(  
    RetentionCriteria.snapshotEvery(numberOfEvents = 100, keepNSnapshots = 2)  
  )  
// alternativ bedingt:  
//   .snapshotWhen((state, event, seqNr) => state.balance % 1000 == 0)
```

Pekko Persistence & CQRS

Wie die Daten gespeichert werden



- Das JDBC-Plugin legt zwei Tabellen an (vereinfacht):
- Schlüssel: `(persistence\_id, sequence\_number)`, Events sind **append-only**
- Separate `event\_tag`-Tabelle verknüpft Events mit Tags für `eventsByTag`

event_journal

persistence_id	sequence_number	event_payload (binär)	write_timestamp	deleted
"Account 42"	1	<Deposited(100)>	1718003...	false
"Account 42"	2	<Deposited(50)>	1718003...	false

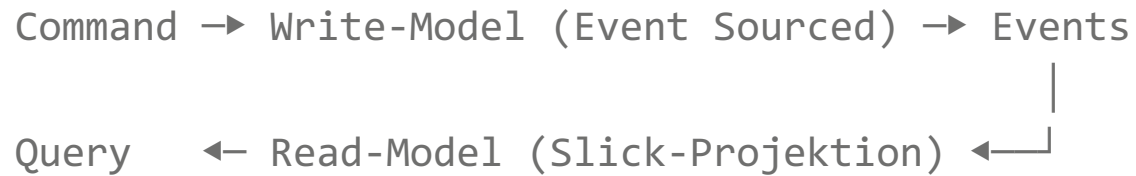
snapshot

persistence_id	sequence_number	snapshot_payload (binär)	created
"Account 42"	2	<State(balance=150)>	1718003...

Pekko Persistence & CQRS

CQRS: Command- & Query-Seite

- **CQRS** = Command Query Responsibility Segregation: Schreiben und Lesen trennen
- **Command-Seite (Write-Model)**: event-sourced, optimiert fürs Verändern (Konsistenz)
- **Query-Seite (Read-Model)**: Projektion der Events, optimiert fürs Lesen (z.B. Slick-Tabelle)
- Die **Events** sind das Bindeglied zwischen beiden Seiten



Pekko Persistence & CQRS

Read-Model per Persistence Query



- `PersistenceQuery` liefert die Events als **Pekko Source** (z.B. `eventsByTag`):
- Hier laufen **alle Themen** zusammen: Persistence (Events) + Streams (Source) + Slick (Read-Model)

```
val readJournal = PersistenceQuery(system)
    .readJournalFor[EventsByTagQuery](journalId)
```

```
val events: Source[EventEnvelope, NotUsed] =
    readJournal.eventsByTag("account", Offset.noOffset)
```

```
events
    .mapAsync(1)(env => updateReadModel(env.event)) // z.B. Slick-Insert/Update
    .runWith(Sink.ignore)
```

- Eine **Projektion** liest Events fortlaufend und aktualisiert das Read-Model
- Der **Offset** merkt sich, bis wohin schon verarbeitet wurde:
- Garantie meist **at-least-once**: ein Event kann doppelt ankommen, also Handler **idempotent** machen
- Ohne Offset würde nach Neustart der ganze Stream erneut verarbeitet

```
event_journal - eventsByTag → Projektion → Read-Model (Slick)
                        |
                        └─> Offset speichern (offset_store)
```

Pekko Persistence & CQRS

Pekko Projections



- Die Library **pekko-projection** übernimmt Offset-Handling, Restart und Delivery-Semantik:
- Der **Offset-Store** ist eine Tabelle, ein Restart setzt am letzten Offset fort

```
val sourceProvider = EventSourcedProvider.eventsByTag[Event](system, journalId, tag = "account")
```

```
class AccountHandler extends Handler[EventEnvelope[Event]] {  
  def process(env: EventEnvelope[Event]): Future[Done] =  
    env.event match {  
      case Deposited(a) => readModel.add(env.persistenceId, a) // Slick-Update  
    }  
}
```

```
val projection = SlickProjection.atLeastOnce(  
  ProjectionId("accounts", "account"), sourceProvider, dbConfig, () => new AccountHandler  
)
```

Pekko Persistence & CQRS

Projektionen im Cluster



- Eine Projektion soll **genau einmal** laufen, sich aber bei Last verteilen
- **ShardedDaemonProcess** verteilt Projektions-Instanzen (nach Tags/Slices) über die Knoten:
- Jede Instanz verarbeitet einen Teil der Events; fällt ein Knoten aus, übernimmt ein anderer
- So skaliert die **Query-Seite** (CQRS) elastisch und ausfallsicher

```
ShardedDaemonProcess(system).init(  
  name = "account-projection",  
  numberOfInstances = tags.size,  
  behaviorFactory = i => ProjectionBehavior(projectionFor(tags(i)))  
)
```

Cluster & Sharding

Cluster & Sharding

Pekko Cluster



- Mehrere Knoten bilden **ein logisches System**, peer-to-peer, **kein** Master
- **Membership** per Gossip-Protokoll + Failure Detection (wer ist erreichbar?)
- Knoten kommen und gehen dynamisch zur Laufzeit
- Erfüllt das Reactive Manifesto direkt:
 - **elastic**: mehr Knoten = mehr Kapazität
 - **resilient**: kein Single Point of Failure
- Basis für **Cluster Sharding**, Cluster Singleton, Distributed Data, Pub-Sub

Cluster & Sharding

Cluster Sharding

- Verteilt **viele zustandsbehaftete Entitäten** (pro id ein Akteur) über die Knoten:
- Pekko übernimmt **Platzierung, Rebalancing, Wiederherstellung**, Adressierung nur per id
- **Passivation**: inaktive Entitäten werden aus dem Speicher entfernt

```
import org.apache.pekko.cluster.sharding.typed.scaladsl.*

val TypeKey = EntityTypeKey[Account.Command]("Account")

val sharding = ClusterSharding(system)
sharding.init(Entity(TypeKey)(ctx => Account(ctx.entityId)))

val ref = sharding.entityRefFor(TypeKey, "acc-42")
ref ! Account.Deposit(100)           // landet auf dem richtigen Knoten
```

Cluster & Sharding

Sharding + Persistence + CQRS



- Die typische Kombination für skalierbare Domänen:
 - jede Entity ist **sharded** (verteilt) **und event-sourced** (persistent)
 - Write-Model skaliert über den Cluster (viele Entitäten, viele Knoten)
 - bei Verschiebung/Neustart: State aus den Events **replayed**
- Das **Read-Model** projiziert die Events clusterweit per Persistence Query (CQRS)
- Ergebnis: **elastic** (skaliert), **resilient** (übersteht Ausfälle), nachrichtengetrieben

viele Entities (sharded, event-sourced) → Events → Read-Model (Projektion)